

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа 2

Выполнил:

Лошкарёв Д. В.

К3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

1. Опишите все функциональные и нефункциональные требования к вашему бэкенд-приложению
2. Выберите формат реализации API, которого вы будете придерживаться: REST API или Backend For Frontend
3. Спроектируйте все эндпоинты вашего API в формате OpenAPI-схемы с учётом реализованной диаграммы БД
4. Опишите тело каждого запроса и тело каждого ответа, продумайте возможные ошибки, возвращаемые из API

Ход работы

1. Проектирование архитектуры API

Для реализации бэкенд-приложения была выбрана архитектура REST. Данный выбор обусловлен необходимостью создания универсального интерфейса для управления ресурсами сервиса (рецептами, пользователями, комментариями). Архитектура построена на использовании стандартных HTTP-методов (GET, POST, PUT, PATCH, DELETE), что обеспечивает предсказуемость взаимодействия и легкость интеграции с различными фронтенд-клиентами.

2. Разработка системы статусов и обработки ошибок

В приложении реализована стандартизированная система кодов ответов HTTP для информирования клиента о результате запроса:

200 OK / 201 Created: Успешное выполнение запроса или создание ресурса.

204 No Content: Успешное удаление без возврата тела ответа.

400 Bad Request: Ошибка валидации входящих данных (например, некорректный формат email).

401 Unauthorized: Запрос требует аутентификации (отсутствует или невалиден JWT-токен).

403 Forbidden: Попытка доступа к чужому ресурсу (например, редактирование рецепта не его автором).

404 Not Found: Запрашиваемый ресурс (рецепт, пользователь или комментарий) не найден.

409 Conflict: Конфликт состояния данных (например, повторный лайк или регистрация на уже занятый email).

3. Эндпоинты

Была спроектирована OpenAPI-схема, включающая следующие группы:

Аутентификация (/auth): Регистрация новых пользователей и выдача JWT-токенов при входе.

Управление рецептами (/recipes):

- Базовый CRUD (создание, чтение, обновление, удаление).
- Поиск и фильтрация по типу блюда и уровню сложности через query-параметры.

Социальное взаимодействие:

- Система лайков (/recipes/{id}/like) с поддержкой идемпотентных запросов.
- Комментарии (/recipes/{id}/comments) для обсуждения блюд.
- Система подписок (/users/{userId}/follow) для формирования ленты авторов.

Личное пространство пользователя:

- Управление профилем (/profile).
- Работа со списком избранного (/favorites), позволяющая сохранять понравившийся контент.

4. Безопасность

Все методы, изменяющие состояние данных (POST, PUT, PATCH, DELETE), защищены механизмом Bearer Authentication (JWT). Права доступа проверяются на уровне бизнес-логики: операции удаления и редактирования доступны только владельцам контента или администраторам.

Вывод

В результате выполнения работы была спроектирована детальная архитектура REST API, соответствующая ранее разработанной схеме базы данных. Использование OpenAPI (Swagger) позволило формализовать контракты взаимодействия между клиентом и сервером, продумать логику обработки исключительных ситуаций и обеспечить безопасность пользовательских данных. Спроектированная система готова к реализации.